



UNIVERSITÀ
DEGLI STUDI
FIRENZE

UNIVERSITÀ DEGLI STUDI DI FIRENZE
DIPARTIMENTO DI INGEGNERIA DELL'INFORMAZIONE

Laboratorio di Optimization Methods

Autore:
Niccolò Scommegna

Corso principale:
Optimization Methods

N° Matricola:
7162797

Docente corso:
Matteo Lapucci

Novembre 2025

Indice

1	Introduzione	2
2	Nozioni Teoriche	2
3	Esperimenti	4
3.1	Numero di Iterazioni	4
3.2	Tempi di Esecuzione	8
3.3	Andamento Funzione Obiettivo e Norme del Gradiente	10
4	Conclusioni	13

1 Introduzione

In questo report vengono presentati i risultati ottenuti durante lo svolgimento del laboratorio di *Optimization Methods*, che ha avuto come obiettivo l'implementazione dell'algoritmo proposto nell'articolo *A Noise-Tolerant Quasi-Newton Algorithm for Unconstrained Optimization* [1] e il suo confronto con altri algoritmi di ottimizzazione non vincolata, come l'algoritmo di discesa del gradiente e l'algoritmo BFGS. L'implementazione è stata effettuata in linguaggio `Python` e i problemi di ottimizzazione sono stati selezionati dalla libreria `PyCUTEst`.

2 Nozioni Teoriche

Gli algoritmi per l'ottimizzazione non vincolata mirano a trovare il minimo di una funzione obiettivo $f : \mathbb{R}^n \rightarrow \mathbb{R}$ utilizzando informazioni sulla funzione stessa, come il gradiente e la matrice Hessiana. In questa sezione vengono descritte le nozioni teoriche fondamentali alla base degli algoritmi implementati nel progetto, in particolare l'algoritmo di discesa del gradiente, l'algoritmo BFGS e l'algoritmo Quasi-Newton tollerante al rumore proposto da Shi et al. [1]. Il metodo di discesa del gradiente utilizza come direzione di discesa l'antigradiente della funzione obiettivo $-\nabla f(x_k)$ e richiede la scelta di uno scalare α_k per determinare la lunghezza del passo. Una pratica comune è la ricerca di α_k tramite line search, che può essere eseguita utilizzando condizioni di Armijo o Wolfe. Queste condizioni garantiscono che il passo scelto riduca sufficientemente il valore della funzione obiettivo e che la variazione del gradiente lungo la direzione di discesa sia compatibile con la curvatura locale della funzione. In notazione tipica, la condizione di Armijo si scrive come:

$$f(x_k + \alpha_k d_k) \leq f(x_k) + c_1 \alpha_k \nabla f(x_k)^T d_k, \quad (1)$$

dove $c_1 \in (0, 1)$ è una costante predefinita e d_k è la direzione di discesa. Per quanto riguarda le condizioni di Wolfe, esse includono sia la condizione di Armijo che una condizione di curvatura. Nel complesso le condizioni forti di Wolfe si esprimono come:

$$f(x_k + \alpha_k d_k) \leq f(x_k) + c_1 \alpha_k \nabla f(x_k)^T d_k, \quad (2)$$

$$|\nabla f(x_k + \alpha_k d_k)^T d_k| \leq c_2 |\nabla f(x_k)^T d_k|, \quad (3)$$

dove $c_2 \in (c_1, 1)$ è un'altra costante predefinita.

L'algoritmo BFGS è un metodo Quasi-Newton che costruisce un'approssimazione della matrice Hessiana B_k basata sui gradienti calcolati nei punti precedenti e la differenza tra i punti delle iterazioni. La matrice B_k viene aggiornata ad ogni iterazione utilizzando la formula di aggiornamento di BFGS:

$$B_{k+1} = B_k + \frac{y_k y_k^T}{s_k^T y_k} - \frac{B_k s_k s_k^T B_k}{s_k^T B_k s_k}, \quad (4)$$

dove $s_k = x_{k+1} - x_k$ e $y_k = \nabla f(x_{k+1}) - \nabla f(x_k)$. Perché l'aggiornamento sia ben definito, e soprattutto per garantire che B_{k+1} sia definita positiva, è necessario e sufficiente che $s_k^T y_k > 0$. Le condizioni di Wolfe forti sono tipicamente impiegate

per assicurare questa proprietà e quindi preservare la definita positività della matrice approssimata della Hessiana ad ogni iterazione. Senza una corretta scelta del passo, che soddisfi tali condizioni, l'aggiornamento potrebbe degenerare e la matrice approssimata potrebbe diventare mal condizionata.

Nell'articolo di Shi et al. [1], si osserva che in presenza di errori (rumore) nella valutazione della funzione e del gradiente, il classico schema BFGS può fallire: differenze di gradiente y_k calcolate su passi troppo piccoli possono essere dominate dal rumore e corrompere l'aggiornamento. Per mitigare questo problema, gli autori propongono modifiche pratiche all'algoritmo BFGS. Gli elementi chiave dell'algoritmo BFGS tollerante al rumore sono:

- **Condizione di controllo del rumore e lengthening:** oltre ad usare le condizioni di Wolfe forti (2, 3) per determinare il passo α_k , viene introdotto il parametro di lengthening (allungamento) $\beta_k \geq \alpha_k$ e richiedono la noise control condition:

$$(\nabla f(x_k + \beta_k d_k) - \nabla f(x_k))^T d_k \geq 2(1 + c_3)\epsilon_g \|d_k\|, \quad (5)$$

dove $c_3 > 0$ è una costante predefinita e ϵ_g è una stima superiore del rumore sul gradiente. Questa condizione garantisce che la differenza osservata nei gradienti sia sufficientemente grande rispetto al rumore e quindi che y_k rifletta la curvatura vera e non solo il rumore. Se α_k non produce un y_k affidabile, l'algoritmo allunga il passo (aumenta β_k) fino a raccogliere una differenza di gradienti significativa.

- **Line search a due fasi tollerante al rumore:** la procedura proposta è una line search a due fasi, nella prima fase si cerca un α_k che soddisfi contemporaneamente le condizioni di Wolfe forti (2, 3) e la noise control condition (5). In questo caso si pone $\beta_k = \alpha_k$. Se non si riesce a trovare tale α_k , si entra nella seconda fase, in cui si cerca un α_k che soddisfi la condizione di Armijo e un $\beta_k \geq \alpha_k$ che soddisfi solo la noise control condition. In questo modo, l'algoritmo garantisce che l'aggiornamento della matrice Hessiana sia basato su informazioni affidabili, anche in presenza di rumore.
- **Aggiornamento della matrice β_k :** una volta determinati α_k e β_k , l'aggiornamento della matrice Hessiana approssimata B_k viene eseguito con la stessa formula dell'algoritmo BFGS (4), ma utilizzando

$$s_k = \beta_k d_k, \quad y_k = \nabla f(x_k + \beta_k d_k) - \nabla f(x_k).$$

Questo assicura che l'aggiornamento sia basato su una differenza di gradienti che supera il rumore, migliorando la stabilità e l'affidabilità dell'algoritmo in contesti rumorosi.

Il contributo principale del paper non è cambiare la struttura fondamentale dell'algoritmo BFGS, ma rendere robusto l'algoritmo esistente in presenza di rumore, introducendo meccanismi per garantire che le informazioni utilizzate per aggiornare la matrice Hessiana siano affidabili. Queste modifiche permettono a BFGS tollerante al rumore di conservare i vantaggi dei metodi Quasi-Newton, anche quando le valutazioni sono affette da errori.

3 Esperimenti

In questa sezione vengono presentati gli esperimenti volti a valutare le prestazioni dell'algoritmo BFGS tollerante al rumore, confrontandolo con BFGS classico (sia nella versione implementata personalmente sia nella versione fornita dalla libreria SciPy) e con il metodo di discesa del gradiente implementato personalmente. Gli esperimenti sono stati eseguiti su un benchmark di sette problemi presi dalla libreria PyCUTEst. Per ciascun metodo è stato registrato: il numero di iterazioni necessarie per la convergenza (definita come norma del gradiente inferiore a un valore tolleranza), l'andamento dei valori della funzione obiettivo e del gradiente ad ogni iterazione, e il tempo di esecuzione totale. Per gli algoritmi progettati per problemi non rumorosi sono state valutate le prestazioni sia su funzioni prive di rumore sia su funzioni rumorose, in modo da evidenziare l'effetto del rumore sugli algoritmi classici e successivamente verificare l'efficacia della versione BFGS tollerante al rumore. Nei paragrafi successivi sono riportati i dettagli e i risultati ottenuti, presentati sia in forma tabellare che grafica.

Prima di passare ai risultati, viene fornita una breve panoramica dei dettagli tecnici degli esperimenti:

- **Ricerca di linea:** per il metodo di discesa del gradiente (GD) è stata implementata una ricerca di linea che utilizza la sola *condizione di Armijo*. Per BFGS classico è stata implementata una ricerca di linea che soddisfa le *condizioni di Wolfe forti*. Per BFGS tollerante al rumore la ricerca di linea è basata sulle stesse Condizioni di Wolfe forti, con l'aggiunta della *condizione di controllo del rumore* descritta nella Sezione 2.
- **Aggiunta di rumore:** ad ogni valutazione della funzione obiettivo e del gradiente è stato aggiunto rumore gaussiano $\mathcal{N}(0, \sigma^2)$, dove σ è il livello di rumore scelto per l'esperimento. Alla valutazione scalare della funzione obiettivo si somma un termine scalare di rumore; al gradiente si aggiunge invece un vettore di rumore, perturbando ciascuna componente.
- **Tolleranza:** una volta fissati i livelli di rumore (in particolare quello sul gradiente), la tolleranza per la condizione di arresto viene determinata a run time affinché risulti sempre maggiore del rumore sul gradiente¹. In questo modo si evita che il rumore impedisca la terminazione degli algoritmi. Inoltre, per garantire la coerenza nel confronto tra metodi, la stessa regola è stata applicata anche alle istanze prive di rumore: la tolleranza è stata calcolata come se fosse presente rumore, mantenendo omogenee le condizioni di arresto tra gli esperimenti.

3.1 Numero di Iterazioni

Per ogni problema scelto, la Tabella 1 riporta il numero medio di iterazioni impiegate dai diversi metodi, sia in assenza che in presenza di rumore di piccola entità

¹La tolleranza dipende principalmente dal valore di rumore applicato e dal numero di variabili del problema.

Tabella 1: Numero medio di iterazioni impiegate dai diversi metodi, sia in assenza che in presenza di rumore di piccola entità (10^{-8}), per ogni problema del benchmark. La tabella riporta anche il numero di esecuzioni che hanno effettivamente raggiunto la convergenza, per ciascuna configurazione sono state eseguite 50 prove. A lato di ogni problema è riportato il numero di variabili o dimensione (Dim).

Problema (Dim)	GD			GD			BFGS			BFGS			BFGS Tollerante			SciPy BFGS			SciPy BFGS		
	no rumore	media	convergeti	no rumore	media	convergeti	no rumore	media	convergeti	no rumore	media	convergeti	no rumore	media	convergeti	no rumore	media	convergeti	no rumore	media	convergeti
ALLINITU (4)	312.000	50	4916.040	31	33.000	50	64.840	50	19.520	50	12.000	50	12.800	1							
BOX (10)	78.000	50	10000.000	0	28.000	50	39.920	48	36.440	50	5.000	0	5.900	0							
BOXPOWER (10)	10000.000	0	10000.000	0	80.000	50	1782.286	45	373.420	49	44.000	0	10.460	0							
BRKMCC (2)	218.000	50	10000.000	0	32.000	50	35.820	44	11.060	50	5.000	50	5.060	7							
BROYDN7D (50)	10000.000	0	10000.000	0	122.000	50	171.880	49	194.660	50	77.000	0	71.200	0							
CURLY20 (100)	10000.000	0	10000.000	0	129.000	50	762.540	48	667.900	50	121.000	0	117.780	0							
DENSCINA (2)	51.000	50	9688.100	2	30.000	50	34.360	50	13.060	50	11.000	50	9.860	0							

(livello di rumore 10^{-8}). La tabella include anche il numero di esecuzioni che hanno effettivamente raggiunto la convergenza. Il numero di esecuzioni per ogni configurazione è stato fissato a 50. La Tabella 2 mostra invece i risultati ottenuti con un rumore di grande entità (livello di rumore 10^{-5}).

I metodi BFGS (sia il classico che la variante tollerante al rumore, oltre alla versione di SciPy) richiedono, come previsto, molte meno iterazioni rispetto al metodo di discesa del gradiente, sia nei test senza rumore sia in quelli rumorosi. L'introduzione del rumore però peggiora le prestazioni complessive: aumenta il numero medio di iterazioni e diminuisce la percentuale di esecuzioni che raggiungono la convergenza.

Il metodo BFGS tollerante al rumore si rivela più robusto di BFGS classico implementato personalmente in presenza di rumore: nella quasi totalità dei test converge con un numero medio di iterazioni inferiore e con un tasso di successo maggiore, ottenendo talvolta risultati comparabili o persino superiori rispetto a BFGS applicato a problemi privi di rumore. Questo suggerisce che le strategie di lengthening, la noise-control e la ricerca di linea adattativa migliorano l'affidabilità e le prestazioni complessive. Al contrario, BFGS classico sotto rumore tende a fallire la ricerca di linea e a generare errori numerici, dovuti a termini come $s_k^T y_k$ e $s_k^T B_k s_k$ nulli o molto piccoli necessari per aggiornare l'approssimazione dell'Hessiana.

La versione del metodo BFGS di SciPy appare la meno influenzata dal rumore in termini di numero medio di iterazioni — è sempre il metodo che impiega meno iterazioni — ma presenta comunque un basso tasso di convergenza — il più basso tra i metodi BFGS — nelle prove rumorose: solo poche esecuzioni raggiungono effettivamente il criterio di arresto in alcuni problemi. Questo comportamento, che si riscontra anche su istanze senza rumore, lascia ipotizzare che SciPy impieghi criteri di arresto o controlli numerici diversi rispetto a quelli usati nelle implementazioni personali (oltre alla sola norma del gradiente minore o uguale di un valore di tolleranza).

Analizzando il numero medio di iterazioni in funzione della dimensione del problema si osserva che, in generale, problemi più grandi richiedono un maggior numero di iterazioni per raggiungere la convergenza, effetto particolarmente evidente nei test senza rumore. Nei test rumorosi invece questa relazione risulta meno marcata: l'influenza del rumore può sovrastare l'effetto della dimensione, rendendo meno prevedibile l'andamento del numero di iterazioni.

Infine, dall'analisi delle due tabelle emerge, in generale, che un aumento del livello di rumore determina una diminuzione del numero medio di iterazioni: ciò è dovuto al fatto che la tolleranza per la condizione di arresto viene calcolata a run time in funzione del rumore, quindi tende ad aumentare con esso, rendendo più "facile" soddisfare il criterio di convergenza. Questa scelta spiega anche le differenze osservate tra i risultati sui problemi senza rumore riportati nelle tabelle, poiché la tolleranza è stata fissata coerentemente con il livello di rumore considerato per ogni esperimento.

Tabella 2: Numero medio di iterazioni impiegate dai diversi metodi, sia in assenza che in presenza di rumore di grande entità (10^{-5}), per ogni problema del benchmark. La tabella riporta anche il numero di esecuzioni che hanno effettivamente raggiunto la convergenza, per ciascuna configurazione sono state eseguite 50 prove. A lato di ogni problema è riportato il numero di variabili o dimensione (Dim).

Problema (Dim)	GD		GD		BFGS		BFGS		BFGS Tollerante		SciPy BFGS		SciPy BFGS	
	no rumore media	convergeti	no rumore media	convergeti	no rumore media	convergeti	con rumore media	convergeti	con rumore media	convergeti	no rumore media	convergeti	con rumore media	convergeti
ALLINITU (4)	226.000	50	427.960	31	22.000	50	26.400	50	15.940	50	10.000	50	11.620	5
BOX (10)	45.000	50	10000.000	0	19.000	50	26.120	50	34.920	50	5.000	50	5.240	0
BOXPOWER (10)	10000.000	0	10000.000	0	49.000	50	2498.755	39	666.380	47	9.000	50	9.760	0
BRKMCC (2)	140.000	50	9719.800	1	21.000	50	25.880	48	12.660	50	4.000	50	3.820	9
BROYDN7D (50)	139.000	50	10000.000	0	104.000	50	137.000	50	179.940	50	71.000	0	68.000	0
CURLY20 (100)	10000.000	0	10000.000	0	120.000	50	1250.020	46	624.240	50	118.000	0	114.000	0
DENSCHNA (2)	31.000	50	2982.200	40	20.000	50	24.480	50	11.280	50	9.000	50	8.200	4

3.2 Tempi di Esecuzione

Per ciascun problema, la Tabella 3 riporta il tempo medio di esecuzione dei diversi metodi, sia in assenza di rumore sia con un rumore di elevata entità (livello 10^{-5}). Per ogni configurazione sono state eseguite 50 repliche. Nell'analisi dei tempi è stato scelto di riportare solo i risultati relativi al rumore elevato, poiché i risultati ottenuti con rumore di entità minore mostrano tendenze analoghe; inoltre non è stato riportato il numero di esecuzioni convergenti perché risulterebbe ridondante rispetto a quanto già esposto nella sezione precedente.

I tempi di esecuzione seguono una tendenza simile a quella osservata per il numero di iterazioni: i metodi BFGS (classico, tollerante al rumore e la versione di SciPy) sono, in generale, più rapidi del metodo di discesa del gradiente sia sui problemi non rumorosi sia su quelli rumorosi. In pochi casi i tempi del gradiente risultano paragonabili — o addirittura inferiori — a quelli dei metodi BFGS (soprattutto quando questi ultimi sono eseguiti su problemi rumorosi): ciò dipende dal fatto che la discesa del gradiente, pur richiedendo più iterazioni, ha un costo per iterazione molto inferiore rispetto ai metodi quasi-Newton, i quali devono aggiornare e memorizzare un'approssimazione dell'Hessiana.

L'introduzione del rumore tende ad aumentare i tempi di esecuzione, coerentemente con l'incremento del numero di iterazioni necessario per raggiungere la convergenza. In alcuni casi i tempi del metodo di discesa del gradiente aumentano in modo significativo in presenza di rumore, a indicare la difficoltà del metodo nel gestire le perturbazioni e il basso tasso di convergenza riportato nella Tabella 2.

Osservando i tempi di esecuzione in funzione della dimensione del problema, emerge che istanze più grandi tendono a richiedere tempi maggiori, in linea con quanto visto per il numero di iterazioni. Questo comportamento è atteso, perché il costo per iterazione cresce con la dimensione del problema, soprattutto per i metodi BFGS, che coinvolgono operazioni matriciali più onerose. Tuttavia, l'effetto della dimensione sui tempi medi di esecuzione non è sempre proporzionale né immediato da leggere, poiché il tempo totale dipende sia dal costo per iterazione sia dal numero di iterazioni necessarie per la convergenza. Nonostante ciò, confrontando i problemi `BROYDN7D` (50 variabili) e `CURLY20` (100 variabili), BFGS classico implementato personalmente mostra un aumento significativo del tempo di esecuzione sul problema più grande, anche se il numero di iterazioni non cresce in modo altrettanto marcato (cfr. Tabella 2).

Infine, anche in termini di tempo di esecuzione, il metodo BFGS tollerante al rumore si mostra più efficiente del metodo BFGS classico (implementato personalmente) quando è presente rumore, confermando la sua maggiore robustezza. La versione di SciPy risulta la più veloce in assoluto, ma, come già osservato, tende ad avere un basso tasso di convergenza nei test rumorosi.

Tabella 3: Tempo medio di esecuzione impiegato dai diversi metodi, sia in assenza che in presenza di rumore di grande entità (10^{-5}), per ogni problema del benchmark. Per ciascuna configurazione sono state eseguite 50 prove. A lato di ogni problema è riportato il numero di variabili o dimensione (Dim).

Problema (Dim)	GD (s)		GD con rumore (s)		BFGS no rumore (s)		BFGS con rumore (s)		BFGS Tollerante con rumore (s)		SciPy BFGS no rumore (s)		SciPy BFGS con rumore (s)	
ALLINTU (4)	0.014334		0.055300		0.002479		0.005629		0.003128		0.001765		0.003193	
BOX (10)	0.003497		1.454260		0.002387		0.005152		0.065018		0.001160		0.002859	
BOXPOWER (10)	0.620002		1.360134		0.004735		0.659949		0.336490		0.001662		0.003148	
BRKMCC (2)	0.010843		1.365621		0.002181		0.006079		0.002479		0.000852		0.001918	
BROYDN7D (50)	0.016969		2.170940		0.027265		0.059845		0.138338		0.014043		0.016338	
CURLY20 (100)	2.095950		2.745580		0.065934		1.310250		0.274606		0.037668		0.041880	
DENSCHINA (2)	0.001707		0.357335		0.001968		0.004736		0.001927		0.001298		0.002671	

3.3 Andamento Funzione Obiettivo e Norme del Gradiente

In questa sezione vengono mostrati alcuni grafici rappresentativi dell'andamento della funzione obiettivo e della norma del gradiente durante l'esecuzione dei diversi metodi. I risultati relativi al metodo di discesa del gradiente non sono stati riportati nelle figure perché, a causa della sua mancata convergenza in molti test, avrebbero reso difficile leggere e confrontare l'andamento degli altri algoritmi.

I grafici si riferiscono al problema del benchmark **BOXPOWER** e mostrano due livelli di rumore: basso (10^{-8}) e alto (10^{-5}), gli stessi impiegati nelle sezioni precedenti. Essi evidenziano l'effetto del rumore sul comportamento degli algoritmi e mettono in luce come il BFGS tollerante al rumore gestisca meglio tali perturbazioni.

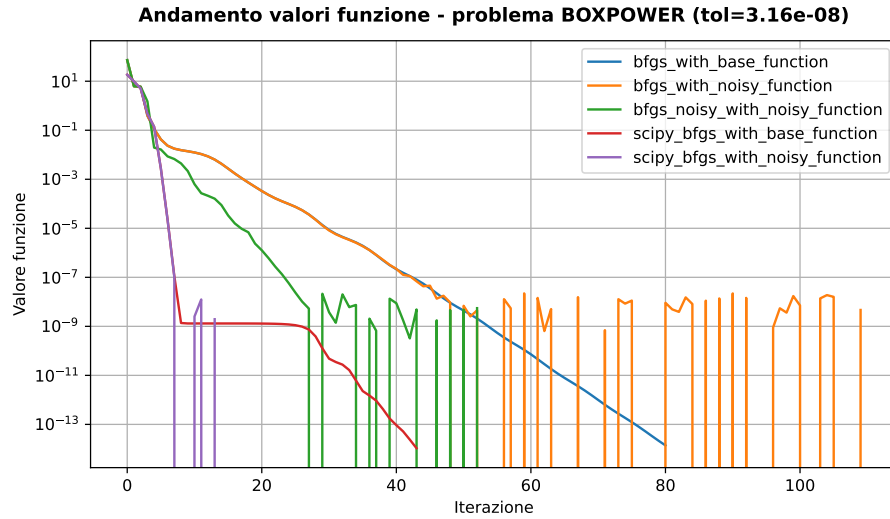
Sull'andamento della funzione obiettivo, in assenza di rumore si osserva la classica diminuzione regolare e monotona verso il minimo per tutti i metodi (vedi Figura 1a). Con l'introduzione del rumore invece emergono oscillazioni più pronunciate, particolarmente marcate nel BFGS classico (cfr. Figura 2a), a indicare che le perturbazioni possono interferire con l'identificazione di direzioni di discesa affidabili.

L'analisi della norma del gradiente conferma queste osservazioni: in presenza di rumore basso (Figura 1b) la riduzione del gradiente è più irregolare rispetto al caso deterministico, ma il BFGS tollerante al rumore mantiene una traiettoria di discesa relativamente stabile rispetto al BFGS classico. Con rumore elevato (Figura 2b) le fluttuazioni del gradiente sono significativamente più ampie per il BFGS classico, mentre la variante tollerante al rumore conserva una tendenza discendente più coerente.

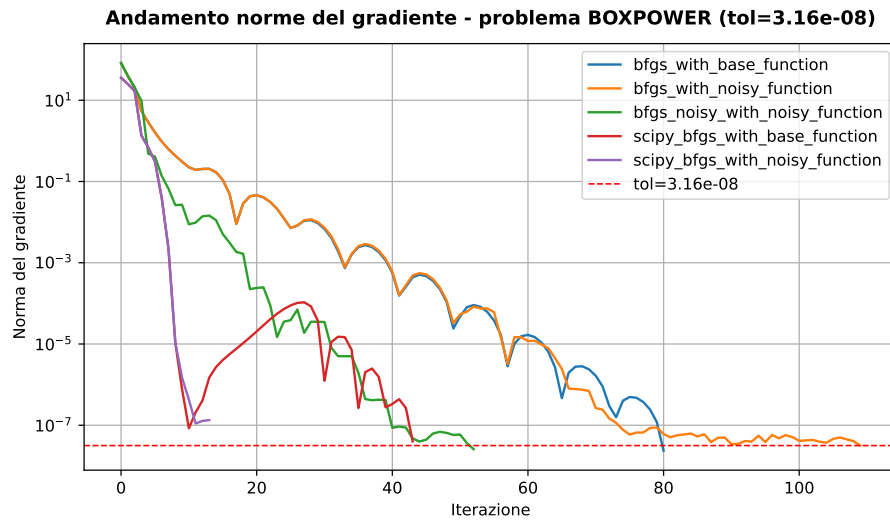
Ancora una volta dai grafici relativi alla norma del gradiente notiamo come il metodo BFGS di SciPy, pur essendo molto veloce, non riesca a soddisfare il criterio di arresto, soprattutto in presenza di rumore, confermando quanto osservato nelle sezioni precedenti. Notiamo però che il metodo riesce a ridurre la norma del gradiente in modo consistente, anche se non fino al valore di tolleranza prefissato.

Infine, dai grafici della norma del gradiente si conferma anche il comportamento della versione SciPy di BFGS: pur essendo molto efficiente in termini di tempo, essa non raggiunge spesso il criterio di arresto nei test rumorosi, benché riduca la norma del gradiente in modo consistente, non sempre arriva al valore di tolleranza prefissato, come già osservato nelle sezioni precedenti.

Nei grafici relativi al problema **BOXPOWER** si osserva che l'aumento del rumore porta a un incremento del numero di iterazioni necessarie per la convergenza. Sebbene questo possa sembrare in contrasto con il comportamento medio evidenziato dalle tabelle relative al numero di iterazioni, una verifica delle tabelle più approfondita per il problema **BOXPOWER** mostra che anche lì non si registra una diminuzione delle iterazioni al crescere del rumore; dunque l'andamento mostrato nei grafici è coerente con i risultati medi riportati in tabella.

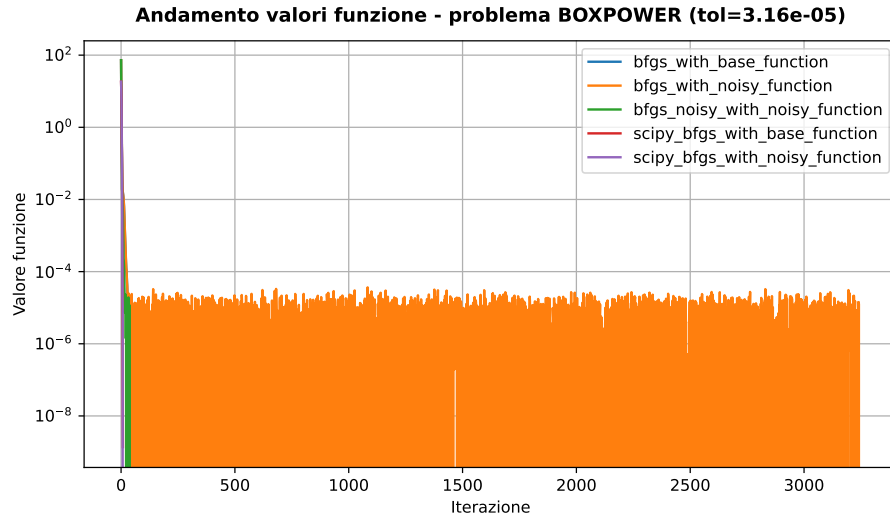


(a) Andamento della funzione obiettivo su BOXPOWER con rumore 10^{-8} .

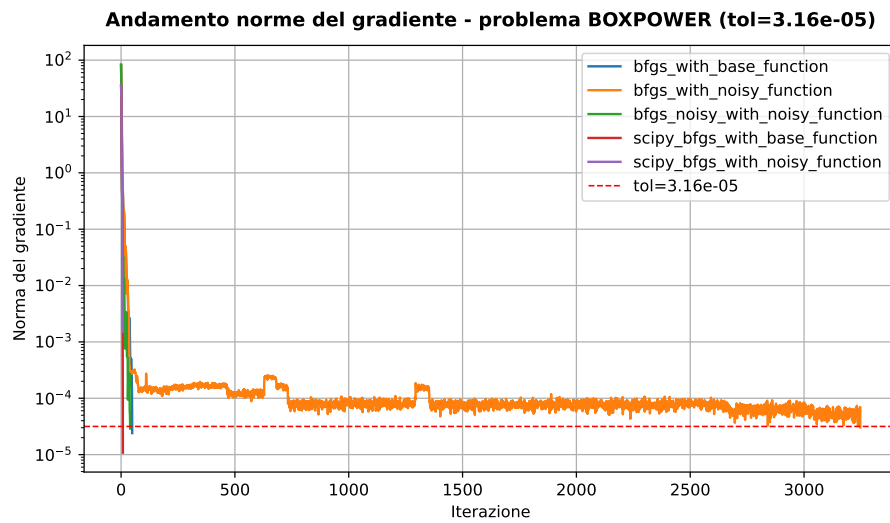


(b) Andamento della norma del gradiente su BOXPOWER con rumore 10^{-8} .

Figura 1: Andamento della funzione obiettivo e della norma del gradiente per i diversi metodi su BOXPOWER con rumore di piccola entità (10^{-8}).



(a) Andamento della funzione obiettivo su BOXPOWER con rumore 10^{-5} .



(b) Andamento della norma del gradiente su BOXPOWER con rumore 10^{-5} .

Figura 2: Andamento della funzione obiettivo e della norma del gradiente per i diversi metodi su BOXPOWER con rumore di grande entità (10^{-5}).

4 Conclusioni

Il confronto sperimentale tra il metodo BFGS tollerante al rumore, BFGS classico (implementato personalmente e in SciPy) e il metodo di discesa del gradiente conferma i vantaggi pratici dei metodi quasi-Newton: i BFGS richiedono infatti molte meno iterazioni rispetto alla discesa del gradiente e, nella maggior parte dei casi, risultano più efficienti in termini di tempo e iterazioni. Tuttavia, l'introduzione di rumore nelle valutazioni della funzione e del gradiente evidenzia differenze rilevanti tra le varianti analizzate.

L'algoritmo BFGS progettato per essere tollerante al rumore si è dimostrato più robusto e affidabile rispetto a BFGS classico: grazie alle strategie di *lengthening* e alla *line-search* adattativa: mantiene l'approssimazione dell'Hessiana definita positiva, converge con maggiore probabilità e con un numero medio di iterazioni inferiore nelle prove rumorose. La versione di SciPy risulta molto efficiente in termini di tempo e di iterazioni ma mostra un tasso di convergenza più basso in presenza di rumore, suggerendo differenze nei criteri di arresto o nelle implementazioni numeriche rispetto alle implementazioni fatte personalmente. Il metodo di discesa del gradiente, pur robusto e semplice, è penalizzato dal maggior numero di iterazioni richieste e dalle difficoltà ad operare efficacemente quando il rumore è rilevante.

In sintesi, l'algoritmo BFGS tollerante al rumore mantiene i vantaggi dei quasi-Newton (rapidità e qualità della direzione di ricerca) adattandoli con successo a scenari rumorosi, rendendolo una scelta promettente quando le valutazioni sono affette da errori o approssimazioni numeriche.

Riferimenti bibliografici

- [1] Hao-Jun M Shi, Yuchen Xie, Richard Byrd, and Jorge Nocedal. A noise-tolerant quasi-newton algorithm for unconstrained optimization. *SIAM Journal on Optimization*, 32(1):29–55, 2022.